CSL7360 Assignment 2: Cost-Efficient Segmentation under a Compute Budget

Track A – classical pipeline (K-Means + graph cut)

1 Summary

The pipeline labels each pixel of a 256×256 photo as main object or background in about 9.0 ms of single-core CPU time per photo (limit 50 ms) with a 38,600-byte parameter file. On DUTS-TE it reaches mIoU 0.630, BF1 0.354, $Q = 0.547$ and CES = 0.527. Everything (K-Means, region model, max-flow, metrics) is written from scratch with NumPy, OpenCV (resize, colour conversion, `filter2D`) and PyTorch (training the logistic regressions, batched metrics).

2 Method

Regions. The photo is shrunk to 64×64 (`INTER_AREA`) and converted to Lab. Each pixel is described by $(L, a, b, w\,x, w\,y)$ with $x, y \in [0, 1]$ and $w = 240$, so clusters are colour-homogeneous *and* compact. My own K-Means (Lloyd's algorithm) starts from a regular 12×12 grid of centres and runs 5 iterations, giving at most $K = 144$ regions (empty clusters are dropped). The nearest-centre search is one matrix product thanks to the identity $\|x - c\|^2 = \|c\|^2 - 2x \cdot c + \|x\|^2$ (the constant $\|x\|^2$ is dropped and $\|c\|^2$ is folded in by appending a column of ones to x).

Region probabilities P_p. Learned only from the training split. For region p I compute 24 features: (i) the position prior (mean over the region's pixels of the logit of the training object-frequency map 64×64); (ii) the log ratio of object/background Lab histograms (16^3 bins) averaged over the region; (iii) contact with each of the four image sides and a touches-border flag; (iv) colour distance to the image border (minimum, mean, and to the mean border colour); (v) global colour contrast and two spatially weighted contrasts (Cheng et al. style), contrast to adjacent regions, distance to the image mean colour, and image-relative versions of two of them; (vi) Lab lightness, chroma, centroid distance to the centre, vertical position, log area; (vii) sharpness: mean |Laplacian| (computed with `filter2D` at 256^2) – in focus objects against blurred backgrounds. These are standardised and, together with their squares, fed to a logistic regression (L-BFGS, soft targets, pixel-count weights). A *second* logistic regression (auto-context) then sees the stage-1 logits and adds: the stage-1 logit propagated between regions of similar colour (two scales) and between nearby similar regions, the neighbour average, an image-specific object-vs-background colour model (kernel density ratio and distance to the mean object/background colour), the predicted object size, and the distance to the object's centre of mass. The result is clipped to $[0.001, 0.999]$. Because the prior and the histogram table are themselves learned from the training photos, the training features of each half of the training split were computed from tables learned on the *other* half (cross-fitting), so the regression does not over-trust them; the final tables use the whole training split.

Graph cut. One node per region plus source and sink. Source$\to p$ has capacity $-\log(1 - P_p)\,|p|$ and $p \to$sink has $-\log(P_p)\,|p|$ ($|p|$ = pixel count). Regions that touch (4-neighbour pixel pairs) are

joined in both directions with capacity $\lambda W_{pq}\,\ell_{pq}$, ℓ_{pq} being the number of touching pixel pairs, and $W_{pq} = \exp(-\|f(p) - f(q)\|^2/2\sigma_d^2)$ on the mean region colours, with $\sigma_d^2 = s \cdot \mathrm{mean}_{pq}\|f(p) - f(q)\|^2 + 10^{-6}$ per image. The minimum cut is found with my own Dinic max-flow (`maxflow.py`); regions reachable from the source in the final residual graph are object. Before the cut, the stage-2 logit z_p is calibrated, $P_p = \mathrm{clip}\big(\sigma((z_p + b)/\tau)\big)$ with offset $b = 0.5$ and temperature $\tau = 1.6$. Validation tuning gave $\lambda = 6$ and $s = 0.25$. The max-flow was checked against brute-force enumeration on 300 random graphs (0 mismatches), and the batched BF1 against a literal pixel-loop implementation (maximum difference 10^{-7}).

Output. The 64×64 region labelling is enlarged to 256×256 bilinearly and thresholded at 0.5 (smoother borders than nearest-neighbour; +0.0014 Q on validation).

3 How the pipeline is kept fast

Table 1 gives the time per stage (wall clock, mean over 1000 DUTS-TE photos, one thread; K-Means dominates). No Python loop runs over pixels; all pixel work is a handful of NumPy/OpenCV calls on a 64×64 image.

- Clustering and features work on 4096 pixels instead of 65 536; only the Laplacian sharpness is computed at full resolution (one `filter2D`, then `INTER_AREA` pooling).
- K-Means: grid initialisation (no random restarts), a single augmented matrix product per iteration, contiguous `float64` rows for `np.bincount` updates (together these changes cut K-Means from 8.6 ms to 3.5 ms per call in an early profile).
- Region statistics, adjacency and shared border lengths come from `np.bincount` on pixel-pair keys; colour distances are 144×144 matrix products.
- Max-flow: before Dinic starts, every region sends $\min(c_s, c_t)$ directly through source→ p →sink, so afterwards each region keeps residual capacity to only one terminal; this leaves a small residual problem (graph cut ≈1.0 ms).
- All learned tables, weights and hyper-parameters are loaded once from one `params.npz`; the file is stored without compression in float32/float64 (38,600 bytes ≈ 0.92% of 4 MiB).
- Single thread: the four thread environment variables are set before NumPy/OpenCV/PyTorch are imported, `cv2.setNumThreads(1)`, `torch.set_num_threads(1)`; no multiprocessing.

stage	ms per photo
Downscale to 64×64 + BGR→Lab	0.40
K-Means (5 iterations, 144 regions)	7.69
Region features, adjacency, colour distances	2.61
Two logistic regressions	1.52
Edge weights + Dinic max-flow	1.04
Paint labels, enlarge to 256×256	0.21
total (sum of stages)	13.47

Table 1: Time per stage. The official T_{CPU} in Table 2 is measured separately with `time.process_time()` around the full pipeline.

4 Experimental setup

Data. DUTS-TR (10 553 photos) was split with a fixed seed into 9,498 training and 1,055 validation photos; all design choices (clustering settings, features, λ, s) were made on the validation

split. DUTS-TE (5 019 photos) was used only for the final evaluation. Photos were resized with `INTER_AREA`, masks with `INTER_NEAREST` and binarised at 128.

Hardware/software. Intel Core i5-12450H (12th Gen, 8 cores, 2.0 GHz base), 16 GB RAM, Windows 11, timed pipeline on one core/one thread; Python 3.13.2, NumPy 2.2.1, OpenCV 4.13.0, PyTorch 2.10.0+cpu (CPU build; training and inference on CPU). T_{CPU} is the total of `time.process_time()` differences around `segment()` over all 5 019 DUTS-TE photos (one at a time, after one warm-up run) divided by 5 019. On this laptop timings vary by a few ms between runs (frequency scaling), so repeated runs of `predict.py` on a single photo scatter around the DUTS-TE average.

5 Results

set	photos	mIoU	BF1	Q	T_{CPU} (ms)	S_{disk} (bytes)	CES
Validation (from DUTS-TR)	1055	0.762	0.489	0.680	9.17	38,600	0.655
DUTS-TE (official)	5019	0.630	0.354	0.547	**8.98**	38,600	0.527
Own photos	8	0.840	0.542	0.750	7.81	38,600	0.723

Table 2: Quality and cost. CES in every row uses the official T_{CPU} measured on DUTS-TE; the T_{CPU} column shows the time measured on that set itself (only the DUTS-TE one is the official figure; `process_time` advances in 15.6 ms ticks on Windows, which is harmless over thousands of photos but makes the figure coarse for a small set).

variant (validation)	mIoU	BF1	Q
stage-1 regression only, no graph cut ($\lambda = 0$)	0.708	0.380	0.609
stage-1 regression + graph cut	0.720	0.438	0.636
two-stage regression, no graph cut	0.740	0.420	0.644
two-stage regression + graph cut	0.751	0.474	0.668
+ probability calibration (final)	**0.762**	**0.489**	**0.680**

Table 3: Ablation. The graph cut adds +0.024 Q, the second logistic regression adds +0.032 Q, the calibration +0.012 Q (all on validation).

Design. The final design is: a 64×64 Lab+position image clustered by K-Means from a 12×12 grid start with 5 iterations ($K \leq 144$ regions); 24 region features; two stacked logistic regressions (the second one sees the stage-1 logits); probability calibration with offset $b = 0.5$ and temperature $\tau = 1.6$; a graph cut with $\lambda = 6$ and $s = 0.25$; and a bilinear enlargement of the 64×64 labelling to 256×256. Its Q is 0.680 on validation, 0.547 on DUTS-TE and 0.750 on the own photos.

Validation vs. DUTS-TE. DUTS-TE is clearly harder than my validation split: its objects are much smaller (mean object area 14.8% vs 27.3% of the photo), so the position prior and "big salient blob" cues learned from DUTS-TR transfer less well. Table 4 shows the object IoU on DUTS-TE by object size.

6 Own photos

Photos. Eight phone photos were taken around the hostel and campus. Each shows one main object that is not tiny (object areas range from 2% to 14%). The set covers indoor and outdoor scenes, a mildly cluttered background (item1, item3), partial occlusion (item3) and hard sunlight

object area	TE photos	% of TE	% of validation	mean object IoU (TE)
0–2%	407	8.1	1.4	0.155
2–10%	1895	37.8	12.3	0.351
10–40%	2365	47.1	64.9	0.474
40–100%	352	7.0	21.3	0.508

Table 4: Object IoU on DUTS-TE by object size (analysis only; nothing was tuned on DUTS-TE).

with cast shadows (item8). It does not cover near/far systematically, and the lighting was not controlled, so no claim is made about tube light versus daylight.

Masks. Photos were resized with the rule of Section 1b. No interactive annotation program could be run in my session, so each mask is a set of hand-written polygons (`annotations.py`; coordinates read off zoomed, gridded views of the resized photo and checked on overlays; thin bicycle tubes are drawn as thick strokes, wheels as filled shapes). Boundaries are accurate to a few pixels, which matters for BF1 (2-pixel tolerance), so the captured BF1 is a pessimistic estimate. The photos were used for testing only. Figures 1 and 2 show every photo, its true mask and the predicted mask as semi-transparent overlays.

photo	object	conditions	area (%)	IoU	BF1
item1	teal backpack	indoor, near; plain wall, rug and a folded cloth with an umbrella on the floor (mild clutter)	10.0	0.872	0.628
item2	bicycle	outdoor, daylight; black bicycle on a lawn in front of a dark hedge	7.7	0.263	0.329
item3	red bottle	indoor; bottle stands behind the laptop screen (partial occlusion), remote and marker beside it	2.1	0.707	0.296
item4	cat	stairwell, daylight; cat in profile on the steps, door frame behind	5.6	0.730	0.434
item5	banana	indoor desk, near; laptop, notebook and cup edge beside it	5.5	0.803	0.759
item6	dustbin	indoor corner, dim; black liner, wall and a pillar behind	10.4	0.794	0.612
item7	red bottle	indoor desk, near; plain wall and table (easy case)	14.4	0.940	0.844
item8	bicycle	indoor, strong sunlight with hard floor shadows; thin tubes and spoked wheels	14.2	0.692	0.434

Table 5: Captured photos: object, conditions and per-photo results (IoU of the object class). Mean object area 8.7% of the photo.

7 Successful cases

Figure 3 shows four DUTS-TE photos that are segmented well. Selection: 12 photos were drawn at random (seed 0) from the 363 TE photos with object IoU ≥ 0.8, and four were picked for their different backgrounds. They are good examples, not typical ones: only 7.2% of the DUTS-TE photos reach IoU ≥ 0.8, 29.7% reach ≥ 0.6 and 34.0% fall below 0.25.

- #**271** – hummingbird against a smooth blue sky.
- #**830** – snowmobile on snow.
- #**3049** – fish on a dark, smooth background.

- #**4239** – stone monument on a plain lawn and sky.

The common pattern is an object with a colour that differs clearly from a smooth, low-texture background and that is large enough to cover many regions: the colour-contrast, border-contact and position cues all agree, and the graph cut then cleans up the boundary.

8 Failure cases

Figure 4 shows four DUTS-TE failures. Selection: 12 photos were drawn at random (seed 0) from the 1706 TE photos with IoU < 0.25, and four of the draw were chosen because each shows a different kind of failure. They are examples, not a ranking of the worst cases.

- #**147** – over-segmentation: a bird in dry grass, 67% of the photo is labelled object (true object 8%).
- #**3820** – tiny object: the sailboat covers 2% of the photo, nothing is predicted.
- #**2134** – poor lighting: a black dog with hard shadows on sunlit grass, nothing is predicted.
- #**4471** – partial detection: only the pale saddle pad is found, the dark horse and rider are missed.

Likely causes. The region model only sees region-level colour, position, contrast and sharpness statistics (these causes are my interpretation of the examples, not separately tested). (1) *Small objects* are covered by only a few of the 144 regions (a region is about 5×5 pixels of the 64×64 image), so one wrong or missed region costs most of the object (object IoU 0.16 below 2% area vs 0.47 for 10–40%). (2) *Over-segmentation*: in low-contrast scenes where the object has the colours of its surroundings (147), the probability calibration offset, which favours the object label and was tuned on DUTS-TR where objects are large, makes the graph cut label most of the photo as object. Several other photos of the same random draw show this flooding (318, 1230: 57% and 66% predicted for objects of 13% and 14%), and it is the mechanism by which the calibration lowers DUTS-TE Q slightly although it raises validation Q. (3) *Poor lighting*: dark objects with hard shadows (2134) merge with the shadows and have little colour contrast, so the colour cues are uninformative. (4) *Partial objects*: the model finds the salient colour part (the pale saddle pad, 4471) rather than the whole object.

Captured photos. Results are uneven ($Q = 0.750$ overall, per-photo object IoU from 0.26 to 0.94, Table 5); with eight photos, this overall number is dominated by a few easy cases and should not be read as better or worse generalisation than DUTS-TE. The strongest results (item7 bottle 0.94, item1 backpack 0.87, item5 banana 0.80, item6 dustbin 0.79) are large, saturated or high-contrast objects on a plain, low-texture background, which is what the colour-contrast and border-contact cues of the model reward. The weakest photo is **item2** (bicycle on the lawn, IoU 0.26): only the central frame and the rear part are found, together with a patch of hedge; the front wheel (thin rim and spokes) is missed. The black bicycle has colours close to the dark hedge, and thin spoked wheels cannot be represented at the 64×64 working resolution. The other three photos of Figure 2 are found completely but slightly enlarged: item8 (bicycle in hard sunlight, IoU 0.69, predicted area 19.2% against 14.2% true), item4 (cat on the stairs, IoU 0.73, 7.5% against 5.6%) and item3 (red bottle behind the laptop, IoU 0.71, 3.0% against 2.1%). These explanations are my reading of the figures and were not tested separately.

Limitations. The approach has no notion of objects or semantics: it can only use colour, position, contrast and focus statistics of ≈144 regions, and it was fitted to DUTS-TR, whose objects are larger than those of DUTS-TE.



Figure 1: Captured photos, photo | true mask | predicted mask.

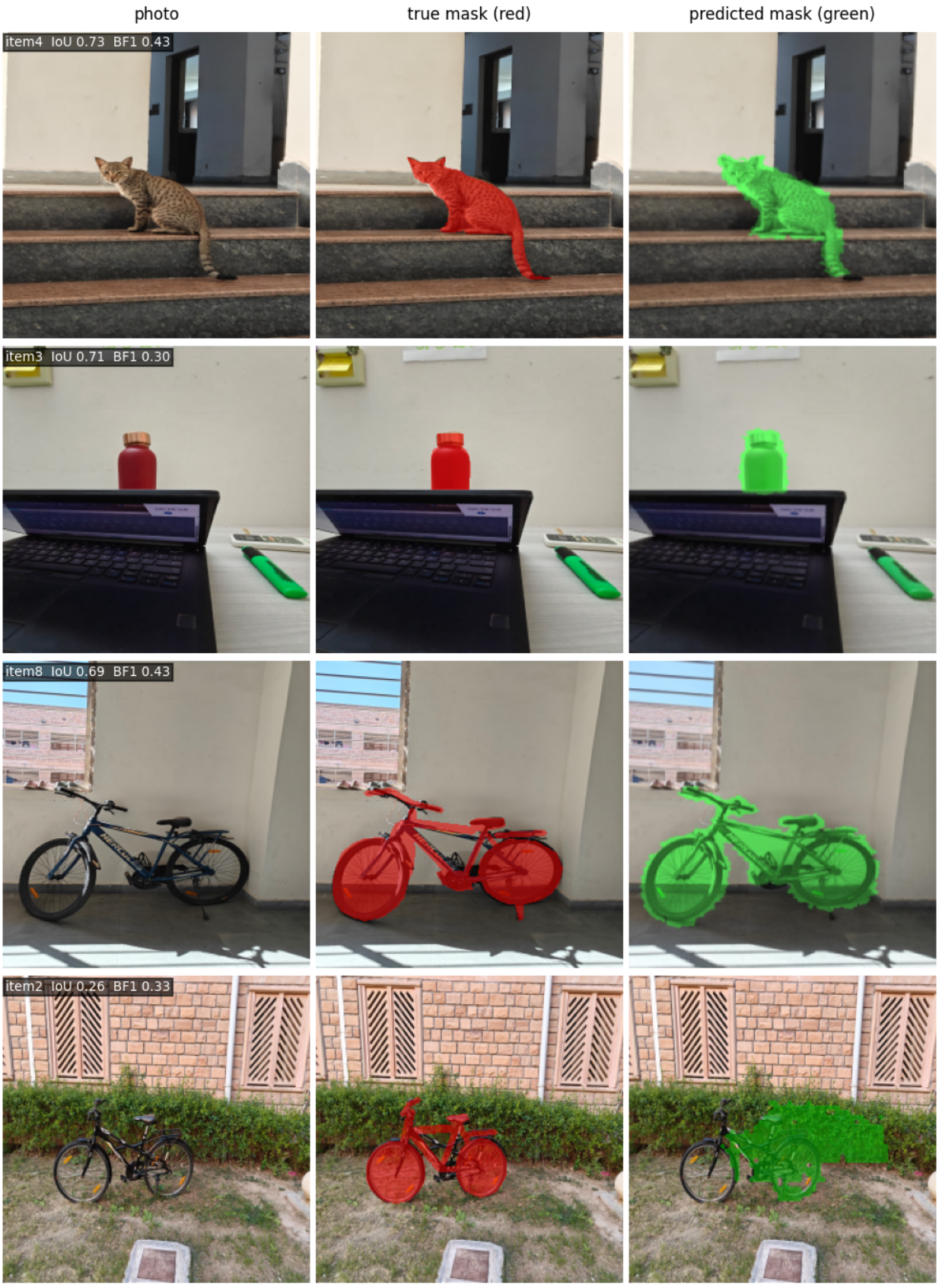


Figure 2: Captured photos, photo | true mask | predicted mask.

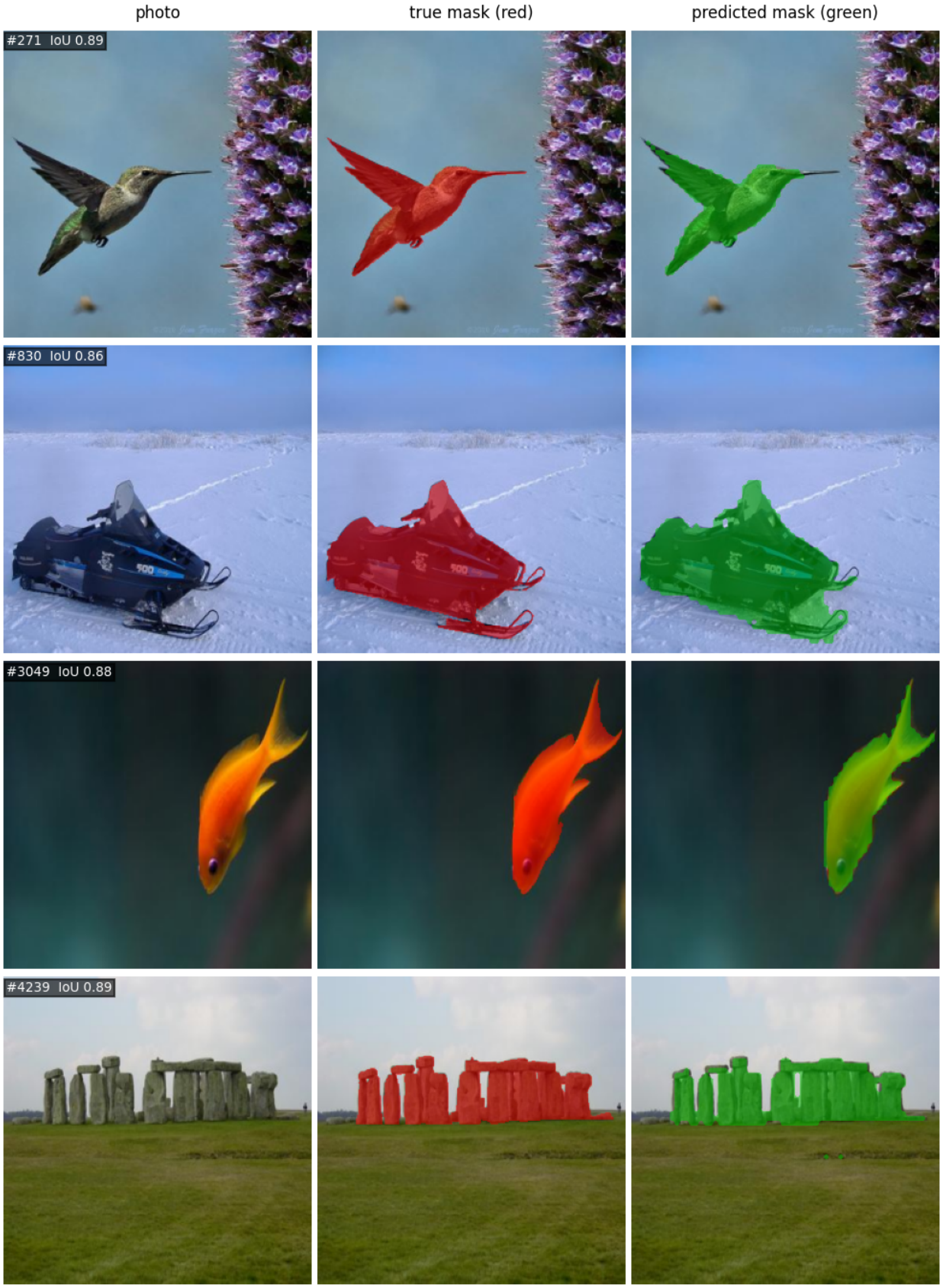


Figure 3: DUTS-TE successes (photo | true mask | predicted mask), IoU in the labels.

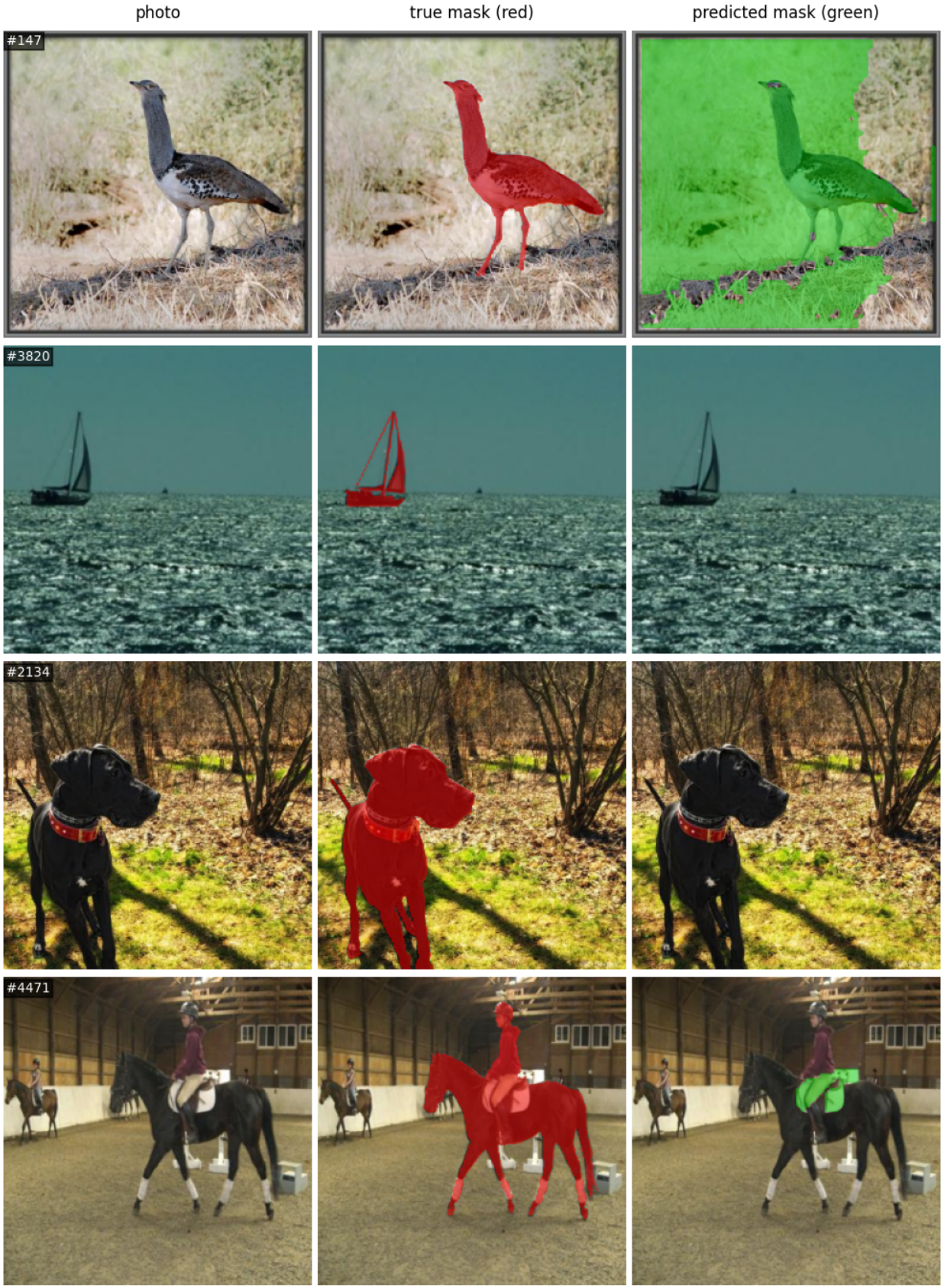


Figure 4: DUTS-TE failures (photo | true mask | predicted mask).